

PINAKA Application Architecture

Complete Notes - Phase 1 / Application v0.1

1. The Big Picture

PINAKA Studio and PINAKA Engine are different things. **PinakaStudio.exe** is the program/product. **PinakaEngine.dll** contains reusable engine technology. Windows starts the Studio executable by entering **main()**; Studio then creates an application object that uses the engine.

```
User opens PinakaStudio.exe | v main() | v StudioApplication | | inherits v pke::Application |  
v Application::run() | +--> initialize engine +--> main loop +--> shutdown engine
```

Terminology: Starting Studio means starting the executable. Initializing PINAKA means initializing the engine and its systems.

2. Why Application Lives in PinakaEngine/Core

The generic Application abstraction belongs to **PinakaEngine/src/Pinaka/Core**. It is an engine concept, not a Foundation utility. Foundation contains low-level reusable building blocks such as containers, memory utilities, math and geometry; Core contains fundamental engine behavior.

```
PinakaEngine/src/Pinaka/ Foundation/ Containers/ Algorithms/ Memory/ Math/ Geometry/ Core/  
Application.h Application.cpp
```

3. Engine Application vs Studio Application

pke represents reusable PINAKA Engine code. Therefore **pke::Application** is the engine's generic application abstraction. **StudioApplication** belongs to PinakaStudio and derives from it.

```
PinakaEngine.dll PinakaStudio.exe pke::Application <----- StudioApplication pke::Window  
Editor UI (later) pke::Renderer Inspector (later) pke::Input Asset Browser (later)
```

Studio-specific classes should not be placed in the pke namespace merely because they use PINAKA.

4. Application Responsibilities

For Application v0.1, the class owns the basic application lifecycle:

- Initialize the engine/lifecycle state.
- Own the main loop.
- Invoke a per-frame application hook.
- Allow a derived application to request shutdown.
- Perform shutdown after the loop ends.

5. Current Application Shape

```
namespace pke { class PK_API Application { public: Application() : m_Running(false) {} virtual  
~Application() = default; void run(); protected: virtual void update(); void requestShutdown();  
private: void initEngine(); void shutdown(); private: bool m_Running; }; }
```

PK_API belongs on **pke::Application** because it crosses the PinakaEngine DLL boundary. **StudioApplication** itself does not need **PK_API** because it belongs to the Studio executable.

6. Lifecycle Implementation

```
void Application::run() { initEngine(); while (m_Running) { update(); } shutdown(); } void
Application::initEngine() { m_Running = true; } void Application::requestShutdown() { m_Running
= false; }
```

The current `initEngine()` and `shutdown()` are intentionally minimal. They are placeholders for real engine subsystem initialization and teardown that will appear as PINAKA grows.

7. Why `update()` Is Virtual

The engine owns the loop, but a derived application must be able to customize what happens each frame. `Application::run()` calls `update()`; because `update()` is virtual, a `StudioApplication` object can dispatch that call to `StudioApplication::update()`.

```
Application::run() | v update() | | virtual dispatch v StudioApplication::update()
```

This is runtime polymorphism used for a real architectural purpose. The base class controls the algorithm/lifecycle; the derived class supplies customizable behavior. This resembles the **Template Method** design pattern.

8. Why `requestShutdown()` Exists

`m_Running` is private because `Application` owns its lifecycle state. `Studio` should not directly modify internal engine state. Instead, a protected `requestShutdown()` function provides a controlled operation.

```
StudioApplication | v requestShutdown() | v m_Running = false | v while condition fails | v
shutdown()
```

Encapsulation rule: Studio can request shutdown; Application owns and changes the running state.

9. Temporary Studio Test

```
void StudioApplication::update() { std::cout << "Update Studio Application" << std::endl;
requestShutdown(); }
```

`requestShutdown()` is currently called immediately only to prove that the complete lifecycle works. In the real engine, shutdown will eventually be triggered by a window-close event.

10. `main.cpp`

```
int main() { StudioApplication app; app.run(); return 0; }
```

`main()` stays intentionally small. `Studio` creates its application and asks it to run; the engine-side `Application` abstraction controls the lifecycle.

11. Full Runtime Trace

```
Windows starts PinakaStudio.exe | v main() | v StudioApplication app | v app.run() | v
Application::initEngine() | v m_Running = true | v while (m_Running) | v virtual update() | v
StudioApplication::update() | v requestShutdown() | v m_Running = false | v shutdown() | v
run() returns | v main() returns
```

12. C++ Concepts Used

Inheritance: `StudioApplication` derives from `pke::Application`.

Runtime polymorphism: `update()` is virtual and overridden by `StudioApplication`.

Virtual destructor: allows safe destruction through an `Application` base pointer.

Encapsulation: `m_Running` remains private and is modified through controlled class behavior.

Access specifiers: run() is public; update() and requestShutdown() are protected customization/control points; internal lifecycle functions and state are private.

DLL API boundary: pke::Application is exported/imported through PK_API.

13. What Happens Next

Application v0.1 is complete enough to move forward. It is not permanently finished. Future systems will naturally connect to it: Window, Events, Time/Delta Time, Input, Renderer and other engine subsystems.

```
User clicks window X | v Operating system / window library | v Window close event | v
Application receives close request | v requestShutdown() | v main loop ends
```

Next architecture problem: design a Window abstraction that acts as PINAKA's bridge to the operating system/windowing library, without coupling high-level Application code unnecessarily to platform details.